6주차 풀이 읽기자료

해설 사용 순서

먼저 학생 실습에서 문제를 독립적으로 풉니다. 이 자료는 힌트와 완전 해설을 포함합니다. 필요한 단계까지만 읽고 다시 풀이합니다.

표시된 예시는 강의에서 만든 예시입니다. 원문과 전체 제약은 각 공식 링크에서 확인합니다.

학생 실습

교사용 진행안

6주차 코드 내려받기

207 · 단계별 힌트

LeetCode 207: Course Schedule · Medium

H1. 아직 남은 선수 조건이 없는 과목은 무엇인가요?

H2. 일부 성분만 검사하면 전체 사이클을 알 수 있나요?

H3. before→after로 간선을 만들고 진입차수 0부터 제거합니다. 제거 수가 V이면 True입니다.

207 · 정당성과 비용

제거 시 남은 선수 조건이 없으므로 선택은 안전합니다. 남은 유향 비순환 그래프에는 진입차수 0 노드가 있어야 합니다. V개를 제거하지 못하면 남은 부분에 사이클이 있습니다.

V=과목 수,E=조건 수. O(V+E) 시간, 인접 리스트 포함 O(V+E) 보조 공간.

207 · 반례와 검증

[[1,0],[0,1]]은 사이클입니다. 상태 1인 정점은 현재 탐색 경로에 있습니다. 그 정점으로 돌아가는 간선을 만나면 사이클입니다. 상태 2인 정점은 탐색이 끝났으므로 결과를 재사용합니다.

```
n=3, prerequisites=[[1,0],[2,1]] → True
```

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

207 · 구현

```
def can_finish(n, prerequisites):
    return len(course_order(n, prerequisites)) == n
```

210 · 단계별 힌트

LeetCode 210: Course Schedule II · Medium

H1. 정답 순서는 유일해야 하나요?

H2. 반환한 순서를 어떻게 알고리즘과 독립적으로 검사하나요?

H3. 진입차수 0 큐에서 꺼낸 순서를 저장하고 마지막에 V개인지 확인합니다.

210 · 정당성과 비용

각 과목은 모든 선수가 제거된 뒤에만 큐에 들어갑니다. 따라서 반환 순서에서 모든 before의 위치가 after보다 앞섭니다. 수가 부족하면 전체 순서가 아닙니다.

V=과목 수,E=간선 수. O(V+E) 시간, graph·indegree·queue·출력 합 O(V+E).

210 · 반례와 검증

[[1,0]]을 1→0으로 읽으면 역순을 만듭니다. 정답 하나와 목록을 직접 비교하지 말고 모든 간선을 검증합니다.

```
n=4, prerequisites=[[2,0],[2,1],[3,2]] → [0,1,2,3]
```

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

210 · 구현

```
def course_order(n, prerequisites):
    graph, indegree = [[] for _ in range(n)], [0] * n
    for after, before in prerequisites:
        graph[before].append(after)
        indegree[after] += 1
    queue = deque(v for v in range(n) if indegree[v] == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)
    return order if len(order) == n else []
```

78 · 단계별 힌트

LeetCode 78: Subsets · Medium

H1. 현재 path를 결과에 넣으면 이후 pop의 영향이 있나요?

H2. 선택한 인덱스보다 앞쪽으로 돌아갈 필요가 있나요?

H3. start 이상에서 하나를 선택하고 다음 인덱스부터 재귀한 뒤 pop합니다. 매 호출에서 path.copy()를 저장합니다.

78 · 정당성과 비용

부분집합마다 오름차순 인덱스 열이 하나 있습니다. start가 그 열만 허용하므로 모든 부분집합을 정확히 한 번 만듭니다. 복원 후 부모의 path는 호출 전과 같습니다.

n=길이. snapshot 포함 $O(n \cdot 2^n)$ 시간·출력 공간, path와 호출 스택 O(n).

78 · 반례와 검증

result.append(path)는 같은 객체를 보관합니다. 마지막 pop 뒤 여러 결과가 빈 목록으로 바뀔 수 있습니다.

`nums=[2,5] → [[],[2],[2,5],[5]]`

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

78 · 구현

```
def subsets(nums):
    result, path = [], []

    def visit(start):
        result.append(path.copy())
        for index in range(start, len(nums)):
            path.append(nums[index])
            visit(index + 1)
            path.pop()

    visit(0)
    return result
```

39 · 단계별 힌트

LeetCode 39: Combination Sum · Medium

H1. 같은 후보를 다시 쓰려면 다음 start는 무엇인가요?

H2. 양수가 아니면 remaining이 계속 감소하나요?

H3. 정렬한 후보에서 같은 index로 재귀해 재사용을 허용하고, remaining보다 큰 후보부터는 중단합니다.

39 · 정당성과 비용

양수 선택마다 remaining이 감소해 종료합니다. 비내림차순 인덱스만 선택하므로 순서만 다른 중복을 피하고 모든 가능한 조합을 표현합니다.

m=후보 수,T=target,c=최소 후보,L=floor(T/c),B=방문 호출 수,S=출력 원소 합. 시간 O(m log m + mB + S), 보조 O(m+L), 출력 O(S). B는 보수적으로 O($(m+1)^{L+1}$).

39 · 반례와 검증

index+1로 재귀하면 [2], target=4의 [2,2]를 놓칩니다. 후보 0은 공식 범위 밖이며 종료 논리를 깨뜨립니다.

```
candidates=[2,5], target=7 → [[2,5]]
```

실행 전에 상태와 반환값을 적고 위의 경계 사례도 확인합니다.

39 · 구현

```
def combinations(candidates, target):
    values, result, path = sorted(candidates), [], []

    def visit(start, remaining):
        if remaining == 0:
            result.append(path.copy())
            return
        for index in range(start, len(values)):
            value = values[index]
            if value > remaining:
                break
            path.append(value)
            visit(index, remaining - value)
            path.pop()

    visit(0, target)
    return result
```

실행과 참고자료

```
cd code
python demo_11.py
python demo_12.py
python verify.py
```

표준 라이브러리 import는 `solutions.py`에 있습니다. LeetCode에서는 공식 메서드 서명에 맞춰 함수를 연결합니다.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

중간 점검의 답과 근거

1. 없습니다. 두 과목이 서로를 기다리므로 빈 목록을 반환합니다.
2. 정답입니다. 0과 1이 모두 2보다 먼저 나오면 됩니다.
3. 4개이며 `[]`도 하나입니다. 경로를 복사해야 이후 pop이 저장된 답을 바꾸지 않습니다.
4. 같은 인덱스 0. 다음 인덱스로 이동하면 필요한 `[2,2]`를 놓칩니다.

개념 질문 해설

DFS의 진행 중 상태 1은 현재 경로에 있으므로 그곳으로 돌아가는 간선은 사이클입니다. 완료 상태 2는 경로에서 빠진 정점입니다. `0→1`, `0→2`, `2→1`에서 완료된 1의 재방문은 사이클이 아닙니다.

재귀의 return은 그 호출만 끝냅니다. 부모는 호출 다음 줄의 pop을 수행하고 for의 다음 선택을 처리합니다. 긴 그래프는 재귀 제한을 피하는 Kahn 구현으로 풉니다.